

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ВЯТСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»

Институт математики и информационных систем
Факультет автоматики и вычислительной техники
Кафедра электронных вычислительных машин

Отчёт по лабораторной работе №2
по дисциплине
«Алгоритмы и структуры данных»

Выполнил студент гр. ИВТб-1303-06-00 _____/Гортоломей И.К./
Проверил заведующий кафедрой ЭВМ _____/Долженкова М.Л./

Киров
2026

Цель работы:

формирование навыков алгоритмического мышления посредством решения набора задач с predetermined характеристиками и уровнем сложности

Номера заданий:

1. <https://codeforces.com/contest/939/problem/A>
2. <https://codeforces.com/contest/1549/problem/B>
3. <https://codeforces.com/contest/2155/problem/B>
4. <https://codeforces.com/contest/292/problem/B>

Ссылка на профиль пользователя:

<https://codeforces.com/profile/GIKExe>

Ссылки на отправления:

1. <https://codeforces.com/contest/939/submission/367858753>
2. <https://codeforces.com/contest/1549/submission/367860887>
3. <https://codeforces.com/contest/2155/submission/367864583>
4. <https://codeforces.com/contest/292/submission/368021035>

Решение заданий

Задание 1: Любовный треугольник

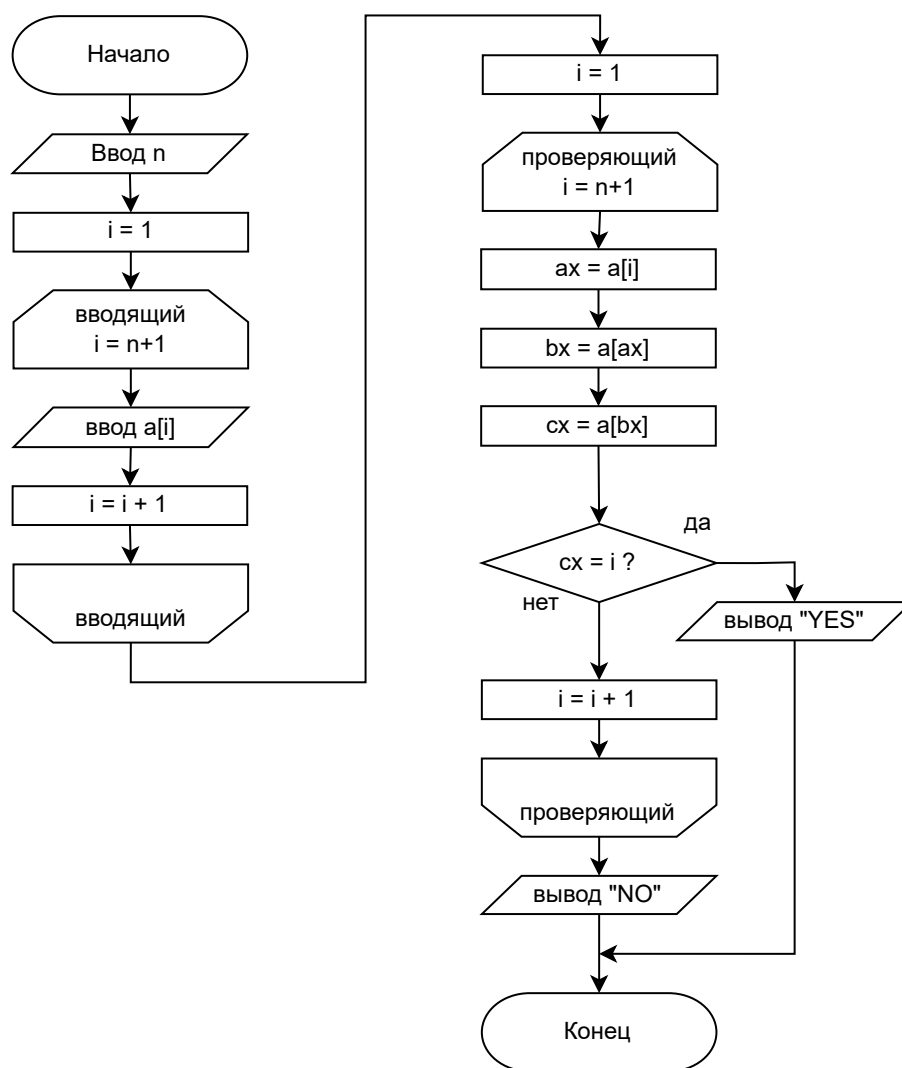


Рис. 1: Схема алгоритма задачи

Листинг 1: Исходный код программы (C)

```
1  #include <stdio.h>
2
3  int main() {
4      int n; scanf("%d", &n);
5      int a[n];
6      for (int i = 0; i < n; i++) scanf("%d", &a[i]);
7      for (int i = 0; i < n; i++) {
8          if (a[a[a[i]-1]-1]-1 == i) {
9              printf("YES");
10             return 0;
11         }
12     }
13     printf("NO");
14     return 0;
15 }
```

Данное задание можно решить просто пройдясь по всем элементам массива и сравнить текущий индекс элемента с индексом который возвращает цепочка из трёх элементов.

Задание 2: Грегор и игра в пешки

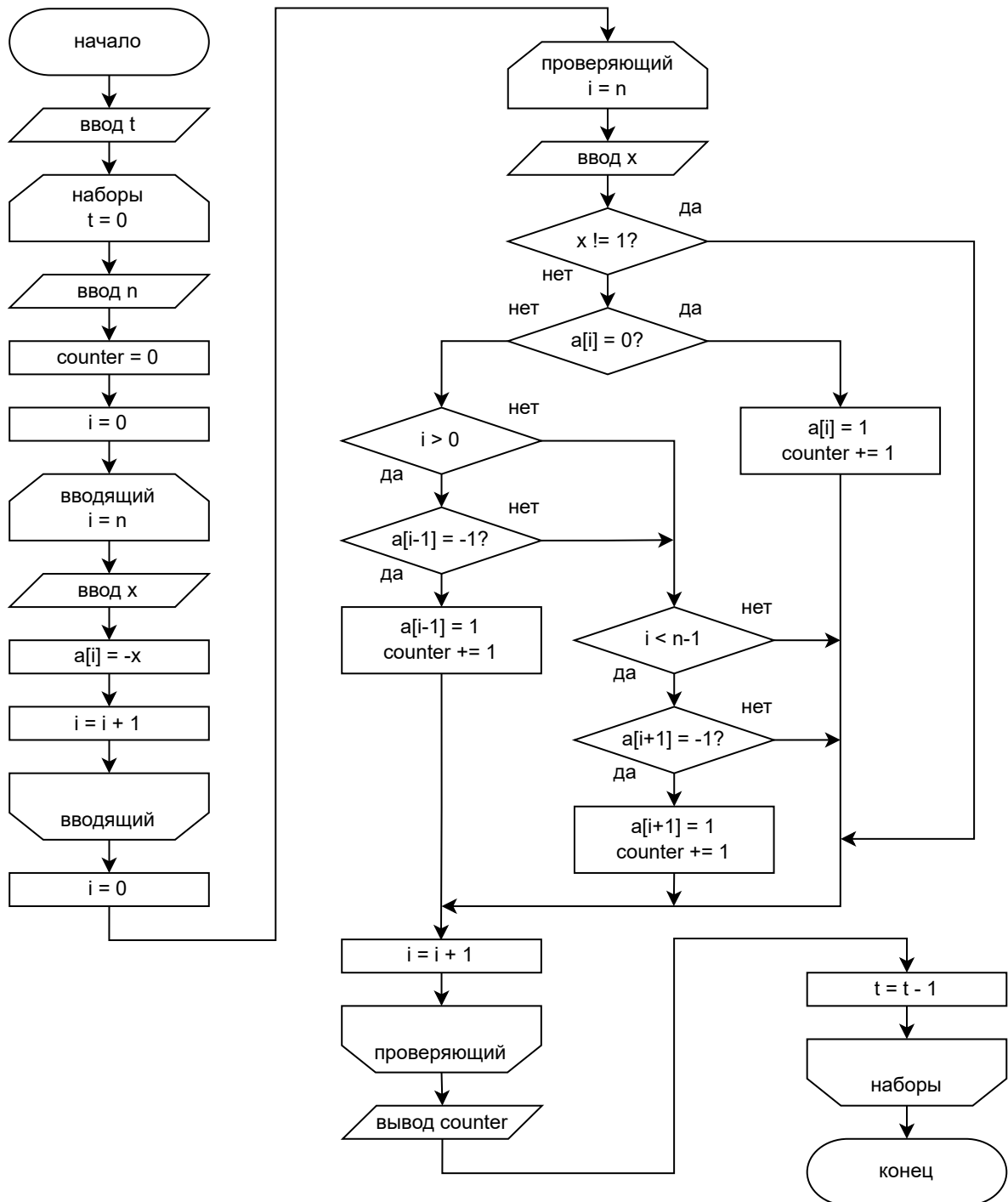


Рис. 2: Схема алгоритма задачи

Листинг 2: Исходный код программы (C)

```
1  #include <stdio.h>
2
3  int main() {
4      int t; scanf("%d", &t);
5      for (; t > 0; t--) {
6          int n; scanf("%d\n", &n);
7          int a[n], counter = 0;
8          for (int i = 0; i < n; i++) a[i] = (getchar()-48) * -1;
9          getchar();
10         for (int i = 0; i < n; i++) {
11             int x = getchar()-48;
12             if (x != 1) continue;
13             if (a[i] == 0) {
14                 a[i] = 1; counter++;
15             } else {
16                 if (i > 0) {
17                     if (a[i-1] == -1) {
18                         a[i-1] = 1; counter++; continue; }
19                 }
20                 if (i < n-1) {
21                     if (a[i+1] == -1) {
22                         a[i+1] = 1; counter++; }
23                 }
24             }
25         }; printf("%d\n", counter);
26     }; return 0;
27 }
```

Это задание можно решить сразу при вводе, проверяя прямой ход пешки или наискосок если там стоит вражеская пешка.

Задание 3: Великий побег Авраама

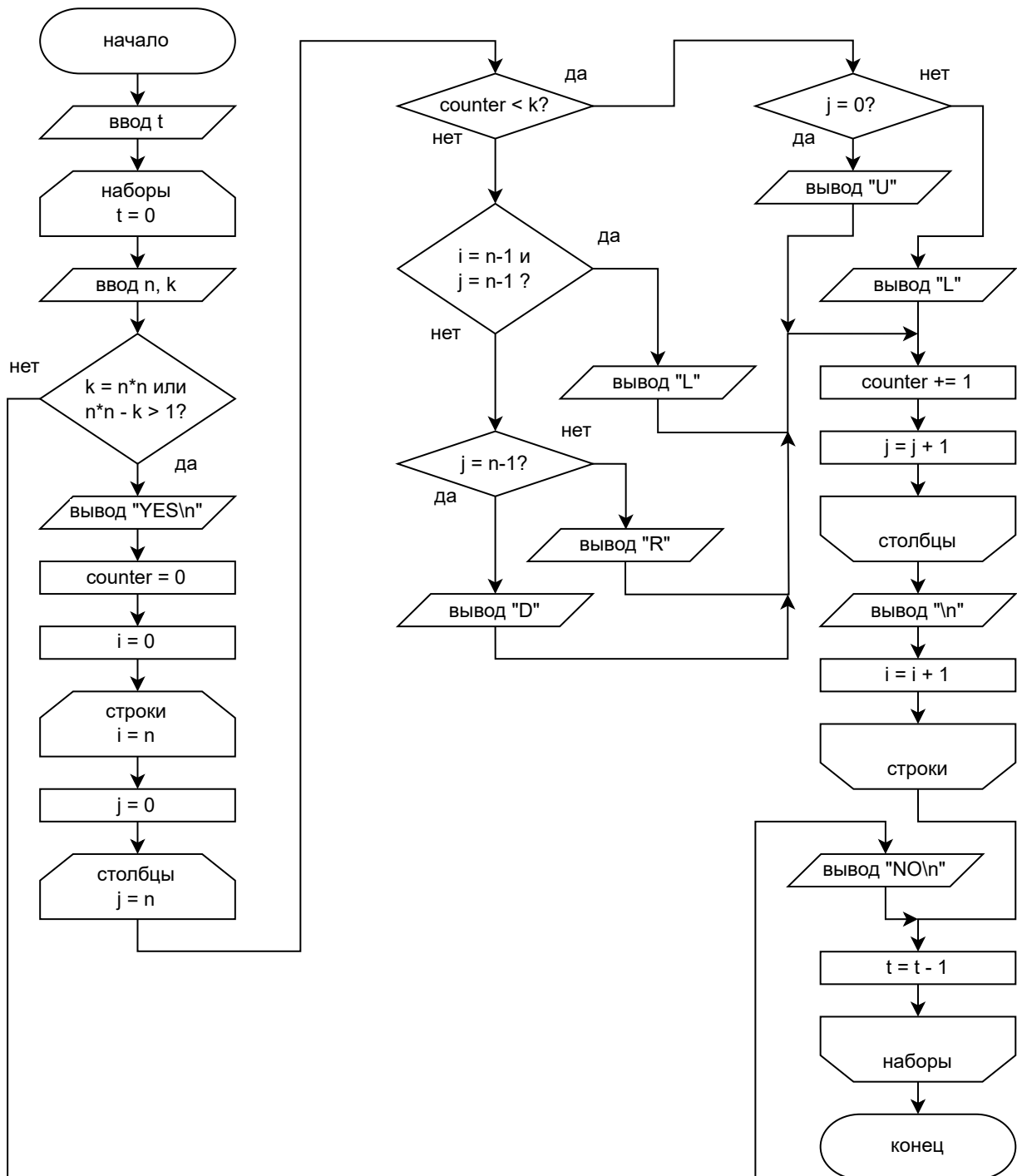


Рис. 3: Схема алгоритма задачи

Листинг 3: Исходный код программы (C)

```
1  #include <stdio.h>
2
3  int main() {
4      int t; scanf("%d", &t);
5      for (; t > 0; t--) {
6          int n, k; scanf("%d %d", &n, &k);
7          if (k == n*n || n*n - k > 1) {
8              printf("YES\n");
9              int counter = 0;
10             for (int i = 0; i < n; i++) {
11                 for (int j = 0; j < n; j++) {
12                     if (counter < k) {
13                         if (j == 0) {printf("U");}
14                         else {printf("L");}
15                     } else {
16                         if (i == n-1 && j == n-1) {printf("L");}
17                         else if (j == n-1) {printf("D");}
18                         else {printf("R");}
19                     }
20                     counter++;
21                 }; printf("\n");
22             }
23         } else printf("NO\n");
24     }; return 0;
25 }
```

Для решения этого задания надо вывести лабиринт в котором k точек старта приводят к выходу, а остальные нет. Для этого можно завести счётчик и сводить к выходу влево-вверх пока счётчик меньше k , а иначе сводить вправо-вниз и развернуть последнюю влево.

Задание 4: Топология сети

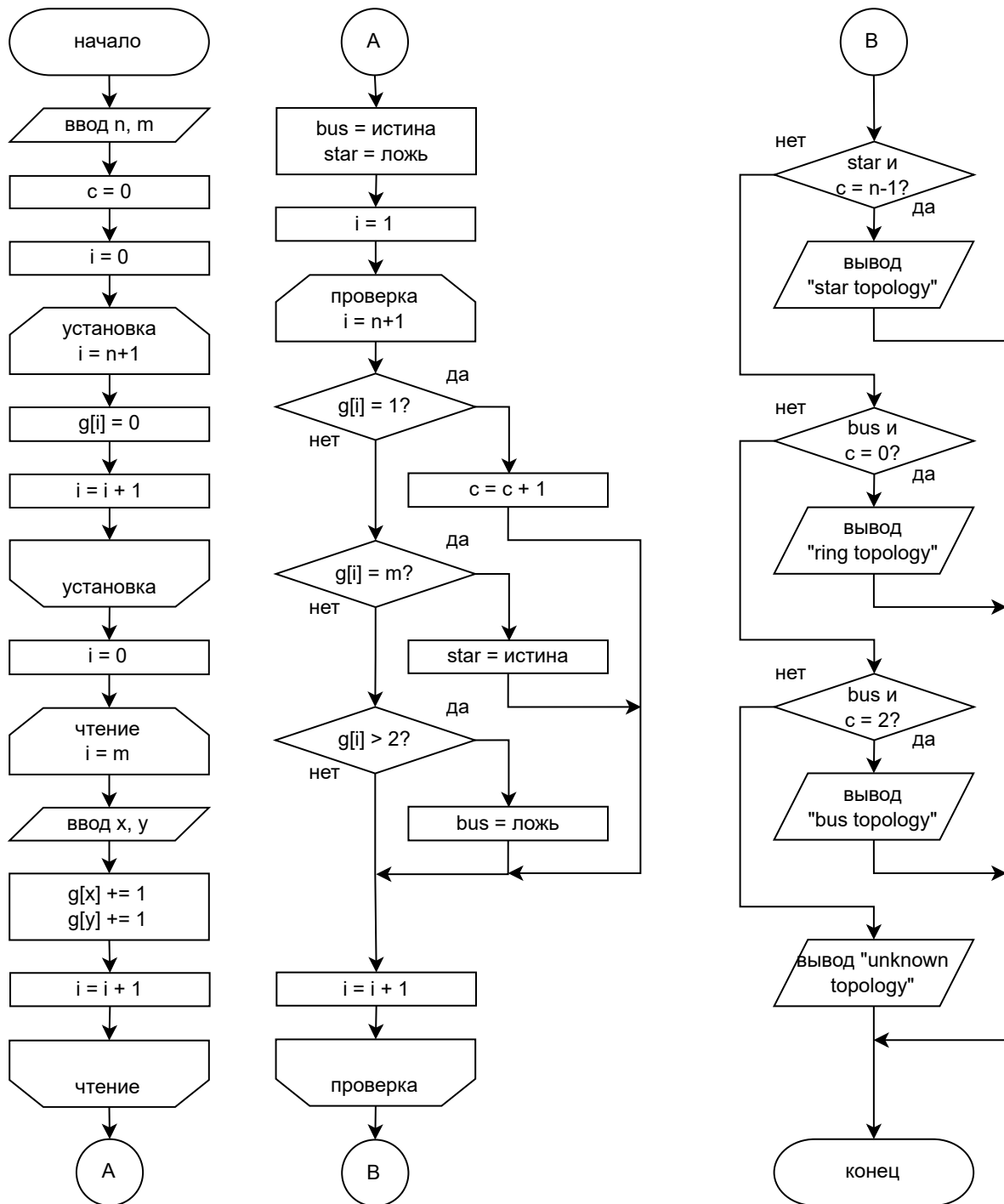


Рис. 4: Схема алгоритма задачи

Листинг 4: Исходный код программы (C)

```
1  #include <stdio.h>
2  #include <stdbool.h>
3
4  int main() {
5      int n, m; scanf("%d %d", &n, &m);
6      int g[n+1], c = 0;
7      for (int i = 0; i < n+1; i++)
8          g[i] = 0;
9
10     for (int i = 0; i < m; i++) {
11         int x, y; scanf("%d %d", &x, &y);
12         g[x]++; g[y]++;
13     }
14
15     bool bus = true;
16     bool star = false;
17     for (int i = 1; i < n+1; i++) {
18         if (g[i] == 1) {c++;}
19         else if (g[i] == m) {star = true;}
20         else if (g[i] > 2) {bus = false;}
21     }
22
23     if (star && c == n-1) {
24         printf("star topology");
25     } else if (bus && c == 0) {
26         printf("ring topology");
27     } else if (bus && c == 2) {
28         printf("bus topology");
29     } else {
30         printf("unknown topology");
31     }
32     return 0;
33 }
```

Это задание легко решается, путём проверки графа на степени вершин, то есть для шины - две вершины имеют степень 1, а остальные степень 2. Для кольца - все вершины имеют степень 2. Так как уже известно что граф связан, то нет необходимости проверять шину и кольцо. Для звезды - одна вершина имеет m степень, а остальные степень 1.

Выводы

В ходе выполнения этой лабораторной работы я занимался решением четырех задач на платформе Codeforces, что позволило мне на практике реализовать навыки алгоритмического мышления. Основной целью было научиться анализировать условия разного уровня сложности и находить для них оптимальные программные решения. Первая задача научила меня работать с индексацией массивов и выстраивать логические цепочки из элементов. Во втором задании я разобрался с алгоритмом движения шахматных пешек, где важно было правильно обрабатывать условия захвата вражеских фигур по диагонали или прямой ход. Третья задача оказалась творческой: я реализовал алгоритм генерации лабиринта, используя систему условий для вывода направлений движения в зависимости от значения счетчика. Наконец, в четвертом задании я применил теорию графов для определения топологии сети. Считая степени вершин, я научился программно различать «шину», «кольцо» и «звезду». Этот опыт помог мне понять, как преобразовывать абстрактные математические модели в работающий код на языке Си. Я научился не просто писать программы, а подбирать наиболее подходящие структуры данных для каждой конкретной ситуации. Процесс отладки и прохождения тестов на Codeforces стал отличной тренировкой внимательности и логики.